Centro Federal de Educação Tecnológica de Minas Gerais
Professora: Poliana Corrêa

RELATÓRIO PRÁTICA 8 - LAOC
CAMINHO DE DADOS DAS INSTRUÇÕES DE TRANSFERÊNCIA DE DADOS

ALUNOS:
GIOVANNI ESTEVAM DOS SANTOS
JOÃO VITOR LOPES FAGUNDES

Belo Horizonte - MG
2025

1. Introdução

A Prática 8 (Caminho de Dados das Instruções de Transferência de Dados) , teve como foco principal integrar a Memória de Dados ao processador e realizar os ajustes e conexões necessárias nos blocos de controle, ULA e banco de registradores. O objetivo é completar o caminho de dados para suportar as instruções de transferência do Formato-I (Tipo-I): Load Word (lw) e Store Word (sw). Para o seu funcionamento, a ULA passa a ser utilizada para calcular o endereço efetivo de memória, somando o valor do registrador base ao campo imediato (estendido de 16 para 32 bits). Com base nos sinais de controle (como MemRead, MemWrite e MemtoReg), o circuito define se o dado será lido da memória para ser escrito no banco de registradores (lw) ou se o valor de um registrador será gravado na memória (sw).

2. Desenvolvimento

Nosso projeto da Prática 8 consiste na expansão do caminho de dados do processador MIPS para suportar as instruções de transferência de dados (**Load Word** e **Store Word**). O projeto focou na integração da **Memória de Dados** e na adaptação da **Memória de Instrução**, além dos ajustes de interconexão no módulo central do processador. A arquitetura agora permite que dados sejam transferidos entre o banco de registradores e a memória, utilizando a ULA para cálculo de endereço efetivo.

Descrição e Decisões de Projeto dos Módulos

Memória de Dados: `memoriaDados`

Descrição: O módulo `memoriaDados` simula a RAM do sistema, responsável por armazenar variáveis e resultados. Ele recebe um endereço de 6 bits, dados de escrita, sinais de controle (`memWrite`, `memRead`) e o clock, fornecendo como saída o dado lido (`readData`).

Decisões de Projeto:

- **Granularidade de Byte:** Para simular fielmente a arquitetura MIPS (que é endereçável a byte), a memória foi declarada como um array de registradores de 8 bits (`reg [7:0] memory [0:63]`). No entanto, como o processador opera com palavras de 32 bits, a lógica de leitura e escrita foi projetada para acessar quatro posições consecutivas de memória simultaneamente (do endereço base até base+3).
- **Temporização de Leitura e Escrita:** Uma decisão importante foi separar as bordas de clock para operações distintas. A escrita ocorre na borda de subida (`posedge clock`), garantindo sincronismo com o resto do sistema , enquanto a leitura ocorre na borda de descida (`negedge clock`). Isso ajuda a estabilizar o sinal `readData` antes que ele seja capturado pelo processador no próximo ciclo.
- **Inicialização Pré-definida:** Para fins de validação do fluxo de dados (conforme o roteiro), o módulo inclui um bloco `initial` que zera a memória e insere o valor 4 explicitamente no endereço 16, permitindo o teste da instrução `lw` logo no início da execução.

Memória de Instrução: `memoriaInstrucao`

Descrição: Este módulo atua como a ROM do processador, recebendo o valor do PC (`readAddress`) e entregando a instrução de 32 bits correspondente.

Decisões de Projeto:

- **Hardcoding do Programa de Teste:** Diferente de uma memória genérica, optou-se por "chumbar" o código de máquina específico da prática dentro do bloco `initial`. As instruções `lw`, `sw` e `add` foram inseridas manualmente em binário byte a byte (ex: `op: 100011` para `lw`). Isso garante que a simulação execute exatamente o algoritmo de teste solicitado: `lw -> sw -> add -> sw -> lw`.
- **Proteção de Endereçamento:** Foi implementada uma lógica de proteção (`outRange`) que verifica se o endereço solicitado ultrapassa o limite físico da memória (124). Caso o PC aponte para fora, a saída é forçada a 0, evitando erros de simulação e leituras de lixo.

Processador (Integração): `processador`

Descrição: Este é o módulo de topo que conecta todos os componentes. Na Prática 8, sua principal função foi integrar a nova memória de dados ao fluxo existente.

Decisões de Projeto:

- **Mapeamento de Endereço da Memória:** Como a memória de dados simulada é pequena (64 posições), decidiu-se conectar apenas os 6 bits menos significativos do resultado da ULA (`ULA_result[5:0]`) à entrada de endereço da memória.
- **Multiplexador de Write Back:** Para suportar a instrução `lw`, foi instanciado um novo multiplexador (`muxDataMem`). Ele utiliza o sinal de controle `MemtoReg` para decidir se o dado a ser escrito no registrador de destino vem da ULA (para instruções tipo R) ou da Memória de Dados (para `lw`).

Banco de Registradores: bancoRegistradores

Descrição: O módulo gerencia os 32 registradores de uso geral do processador. Ele permite a leitura assíncrona de dois registradores (`Read1`, `Read2`) e a escrita síncrona em um registrador (`WriteReg`) quando habilitada pelo sinal `RegWrite`.

Decisões de Projeto:

- **Inicialização Automática:** Implementou-se uma lógica de reset específica que, através de um laço `for`, inicializa cada registrador com o valor correspondente ao seu próprio índice (ex: o registrador 1 recebe o valor 1). Isso foi fundamental para garantir que os registradores usados como base de endereço (ex: `$16`) já contenham valores válidos para os testes de `lw` e `sw` sem necessidade de instruções prévias de carga.
- **Proteção do Registrador Zero:** A lógica de escrita inclui uma verificação condicional explícita (`if ... WriteReg != 5'b0`). Isso garante que, mesmo que uma instrução tente escrever no endereço 0, a operação seja bloqueada pelo hardware, mantendo a constante zero inviolável conforme a arquitetura MIPS.

Topo da FPGA: pratica8_top

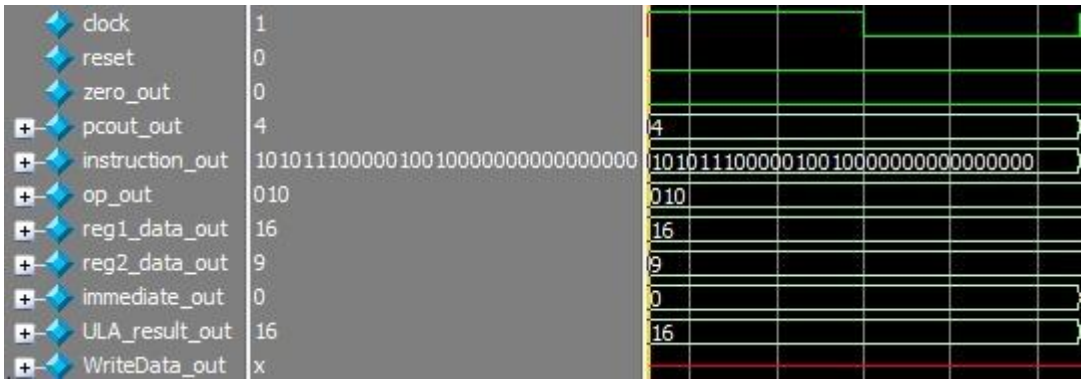
Descrição: Este módulo representa a interface física do projeto na placa DE2, conectando as saídas do processador aos LEDs e displays de 7 segmentos para visualização e depuração.

Decisões de Projeto:

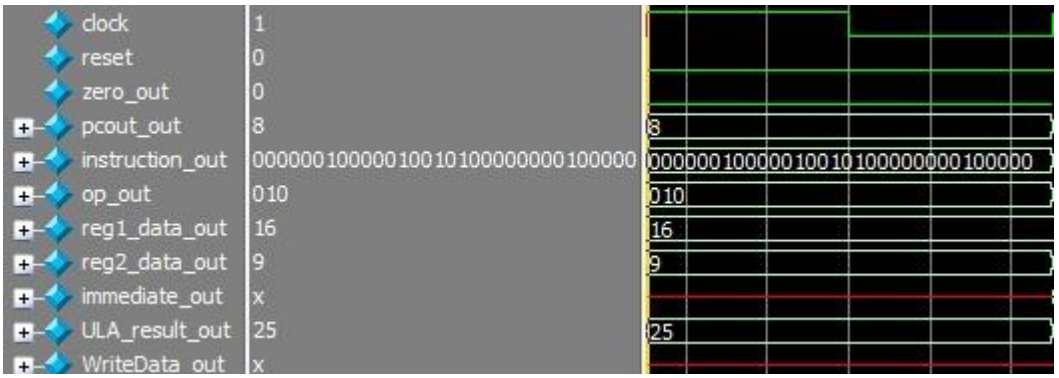
- **Visualização Dinâmica de Operandos:** Para facilitar o entendimento do fluxo de dados, o display HEX4 foi programado com um multiplexador visual controlado por `ULASrc`. Ele exibe o valor do Imediato quando a instrução utiliza essa constante, ou o valor do Registrador 2 caso contrário, permitindo ver exatamente o que está entrando na ULA.
- **Monitoramento de Carga da Memória:** O display HEX7 foi configurado para monitorar a operação de `lw`. Ele exibe o dado vindo da memória (`WriteData_out`) apenas quando o sinal `MemRead` está ativo. Se não houver leitura de memória, a lógica atribui um valor indefinido (`32'bx`) para sinalizar visualmente que aquela via de dados está inativa naquele ciclo.

3. Simulações

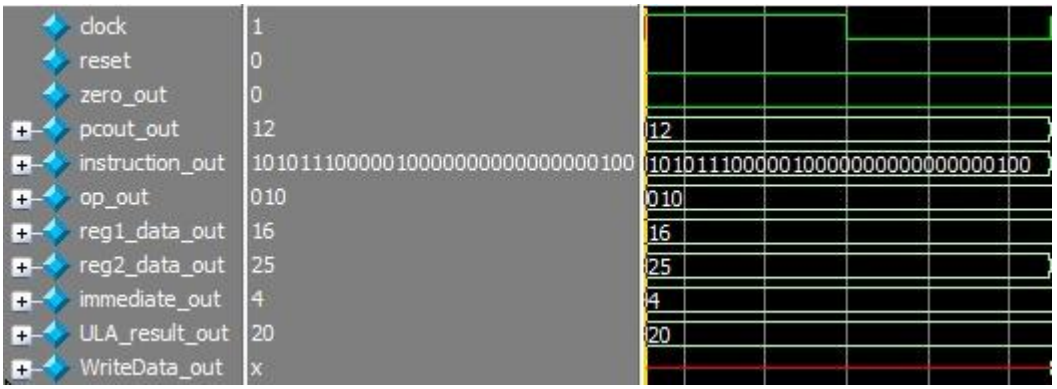
No primeiro ciclo de clock, a operação lw $8, 0($16) é realizada. O registrador 1 ($16) armazena o valor 16, bem como o registrador 2 ($8) armazena o valor 8. A ULA faz a soma do reg1 com o imediato (0) e mostra 16. Como a operação lw possui o sinal MemRead = 1, WriteData mostra o conteúdo armazenado no endereço 16 da memória, que foi inicializado anteriormente com o valor 4.



No segundo ciclo de clock, a operação realizada é sw $9, 0($16), armazenando o valor 9 no endereço 16 da memória, que anteriormente possuía o valor 4. Como MemRead = 0, WriteData apresenta X.



No terceiro ciclo de clock, a operação realizada é add $8, $16, $9. Como MemRead = ULASrc = 0, os fios WriteData e immediate exibem X. O resultado da operação é armazenado em ULA_result = $8 = 16 + 9 = 25.



No quarto ciclo de clock, a operação realizada é sw $8, 4($16). MemWrite = 0, então WriteData = X. ULA_result calcula o endereço 4 + 16 = 20 e então armazena o valor do registrador $8 (que agora é 25, devido à adição feita anteriormente) no endereço 20 da memória.

No quinto ciclo de clock, a operação realizada é lw $9, 4($16). Como MemRead = 1, WriteData mostra o conteúdo do endereço calculado pela ULA (16 + 4 = 20). A instrução anterior armazenou o valor 25 no endereço 20, que só será lido no clock de descida (já que a memória faz a escrita na subida e a leitura na descida). Esse valor então reescreverá o registrador $9.